Podstawy AI – Sterownik wentylatora w języku FCL

FCL (*Fuzzy Control Language*, norma IEC 61131-7) to standardowy, tekstowy format opisu sterowników rozmytych, używany w przemyśle. Plik FCL zawiera wszystko, co sterownik wie: zmienne, funkcje przynależności, reguły i ustawienia operatorów.

Potrzebne pliki (w tym samym folderze co notebook; w Google Colab: ikona folderu po lewej → „Prześlij"):

- `wentylator.fcl` – sterownik do analizy,
- `fcl_narzedzie.py` – narzędzie, które czyta pliki FCL i liczy wynik.

Nic nie trzeba instalować – narzędzie używa tylko `numpy` i `matplotlib`, które są w Colab.

Plan:

1. Analiza pliku FCL (A–C)
2. Dodanie stanu `very_hot` i nowej reguły
3. Testy sterownika
4. Trzy konfiguracje: implikacja, wyostrzanie, suma zbiorów
5. Ocena sterownika
6. Drugie wejście: wilgotność

Do oddania: pliki FCL (co najmniej 3 konfiguracje i wersja z wilgotnością) oraz ten notebook zapisany jako PDF (Plik → Drukuj → Zapisz jako PDF).

```
from fcl_narzedzie import wynik, tabela, rysuj_zbiory, krzywa, wariant, rozszerz, krzywe
print(open("wentylator.fcl", encoding="utf-8").read())
```

```
(* Podstawy AI – sterownik rozmyty wentylatora
   Jedno wejście: temperatura, jedno wyjście: prędkość wentylatora. *)

FUNCTION_BLOCK wentylator

VAR_INPUT
    temperature : REAL;     (* temperatura w pomieszczeniu [°C] *)
END_VAR

VAR_OUTPUT
    fan_speed : REAL;       (* prędkość wentylatora [% mocy] *)
END_VAR

FUZZIFY temperature
    RANGE := (0 .. 40);
    TERM cold := (0, 1) (10, 1) (20, 0);
    TERM warm := (10, 0) (20, 1) (30, 0);
    TERM hot  := (20, 0) (30, 1) (40, 1);
END_FUZZIFY

DEFUZZIFY fan_speed
    RANGE := (0 .. 100);
    TERM low     := (0, 1) (20, 1) (45, 0);
    TERM medium  := (25, 0) (50, 1) (75, 0);
    TERM high    := (55, 0) (80, 1) (95, 0);
    TERM maximum := (85, 0) (100, 1);
    METHOD : COG;           (* wyostrzanie: środek ciężkości *)
    DEFAULT := 0;           (* wynik, gdy żadna reguła nie działa *)
END_DEFUZZIFY

RULEBLOCK reguly
    AND  : MIN;             (* "i" w części JEŚLI *)
    ACT  : MIN;             (* implikacja: MIN przycina, PROD skaluje *)
    ACCU : MAX;             (* suma zbiorów na wyjściu: MAX, BSUM *)

    RULE 1 : IF temperature IS cold THEN fan_speed IS low;
    RULE 2 : IF temperature IS warm THEN fan_speed IS medium;
    RULE 3 : IF temperature IS hot  THEN fan_speed IS high;
END_RULEBLOCK

END_FUNCTION_BLOCK
```

---

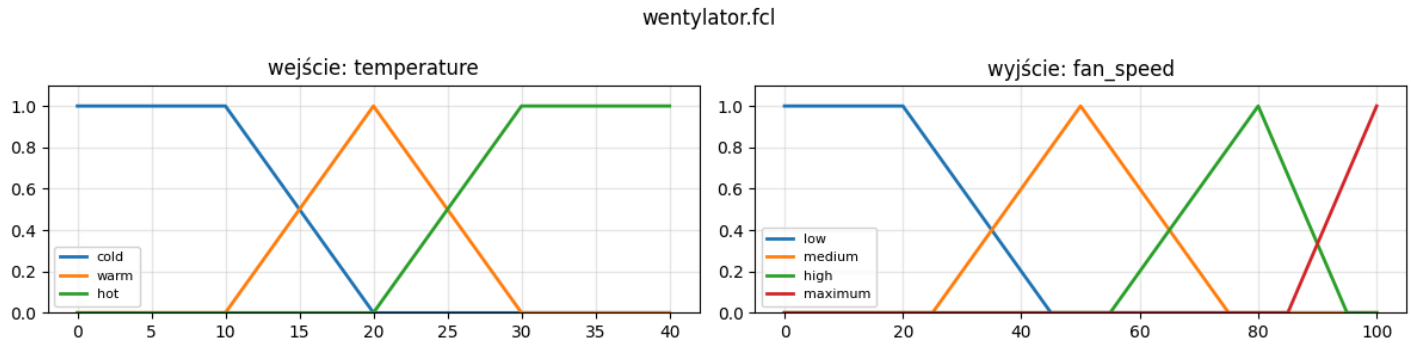
1. Analiza pliku FCL

Plik składa się z bloków. Każdy odpowiada jednemu krokowi sterownika rozmytego:

blok	co zawiera
`VAR_INPUT` / `VAR_OUTPUT`	nazwy wejść i wyjść
`FUZZIFY`	funkcje przynależności wejścia, zapisane punktami `(x, μ)` – łamana przez te punkty
`DEFUZZIFY`	funkcje przynależności wyjścia, metoda wyostrzania `METHOD`, wartość `DEFAULT`
`RULEBLOCK`	operatory (`AND`, `ACT` – implikacja, `ACCU` – suma zbiorów) i reguły `IF … THEN …`

Przykład: `TERM cold := (0, 1) (10, 1) (20, 0);` znaczy: do 10 °C „zimno" w stopniu 1, potem stopień maleje liniowo do 0 przy 20 °C.

```
rysuj_zbiory("wentylator.fcl")
```



Zadanie 1A. Wypisz zmienne wejściowe i wyjściowe (nazwa, zakres, jednostka).

Zadanie 1B. Wypisz funkcje przynależności każdej zmiennej i opisz słowami, co znaczą (np. „cold – do 10 °C na pewno zimno, od 20 °C na pewno nie”).

Zadanie 1C. Wypisz reguły i opisz jednym zdaniem, jak działa ten sterownik.

Zadanie 1A Wejscie: temperature, zakres 0 do 40, jednostka stopnie Celsiusza Wyjscie: fan_speed, zakres 0 do 100, jednostka procent mocy

Zadanie 1B cold: do 10 stopni na pewno zimno (przynaleznosc 1), od 20 stopni na pewno nie (przynaleznosc 0) warm: od 10 stopni rośnie do 1 przy 20 stopniach, potem spada do 0 przy 30 stopniach hot: od 20 stopni rośnie do 1 przy 30 stopniach i zostaje 1 do 40 stopni low: pelna przynaleznosc do 20%, spada do 0 przy 45% medium: trojkatna funkcja, szczyt przy 50%, zakres od 25% do 75% high: trojkatna funkcja, szczyt przy 80%, zakres od 55% do 95% maximum: rośnie od 85% do pelnej przynaleznosci przy 100%

Zadanie 1C Reguly:

1. Jesli temperature jest cold, to fan_speed jest low
2. Jesli temperature jest warm, to fan_speed jest medium
3. Jesli temperature jest hot, to fan_speed jest high Opis dzialania: Sterownik płynnie zwieksza predkosc wentylatora wraz ze wzrostem temperatury w pomieszczeniu na podstawie trzech prostych regul.

---

2. Dodanie stanu `very_hot`

Komórka poniżej zapisuje plik `wentylator_k1.fcl`. Na start zawiera kopię oryginału. Zmień go bezpośrednio w komórce:

1. **(A)** Dodaj do `temperature` stan `very_hot`. Pamiętaj, że `hot` i `very_hot` muszą się **nakładać**, a `hot` powinien przestać być płaski do 40 °C (inaczej przy 40 °C będzie jednocześnie w pełni „hot” i „very_hot”).
2. **(B)** Dodaj regułę: jeśli `very_hot`, to `fan_speed` = `maximum`.

Uruchom komórkę – zapisze plik. Potem uruchom następną, żeby zobaczyć wykres.

```
%%writefile wentylator_k1.fcl
(* TODO: dodaj stan very_hot i regułę dla niego *)

FUNCTION_BLOCK wentylator

VAR_INPUT
    temperature : REAL;     (* temperatura w pomieszczeniu [°C] *)
END_VAR

VAR_OUTPUT
    fan_speed : REAL;       (* prędkość wentylatora [% mocy] *)
END_VAR

FUZZIFY temperature
    RANGE := (0 .. 40);
    TERM cold := (0, 1) (10, 1) (20, 0);
    TERM warm := (10, 0) (20, 1) (30, 0);
    TERM hot  := (20, 0) (30, 1) (36, 0);
    TERM very_hot := (30, 0) (40, 1);
END_FUZZIFY

DEFUZZIFY fan_speed
    RANGE := (0 .. 100);
    TERM low     := (0, 1) (20, 1) (45, 0);
    TERM medium  := (25, 0) (50, 1) (75, 0);
    TERM high    := (55, 0) (80, 1) (95, 0);
    TERM maximum := (85, 0) (100, 1);
    METHOD : COG;           (* wyostrzanie: środek ciężkości *)
    DEFAULT := 0;           (* wynik, gdy żadna reguła nie działa *)
```

```
END_DEFUZZIFY

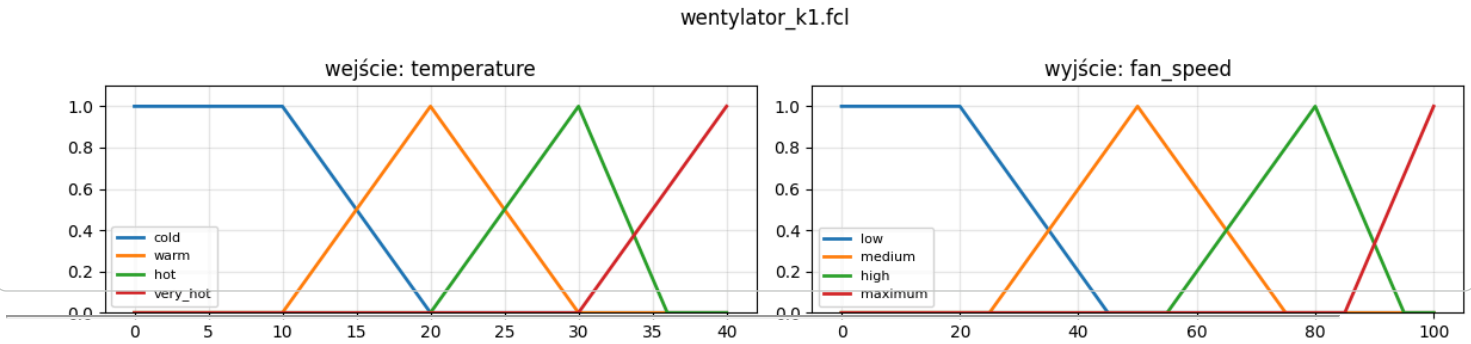
RULEBLOCK reguly
    AND  : MIN;             (* "i" w części JEŚLI *)
    ACT  : MIN;             (* implikacja: MIN przycina, PROD skaluje *)
    ACCU : MAX;             (* suma zbiorów na wyjściu: MAX, BSUM *)

    RULE 1 : IF temperature IS cold THEN fan_speed IS low;
    RULE 2 : IF temperature IS warm THEN fan_speed IS medium;
    RULE 3 : IF temperature IS hot  THEN fan_speed IS high;
    RULE 4 : IF temperature IS very_hot THEN fan_speed IS maximum;
END_RULEBLOCK

END_FUNCTION_BLOCK
```

```
Overwriting wentylator_k1.fcl
```

```
rysuj_zbiory("wentylator_k1.fcl")
```

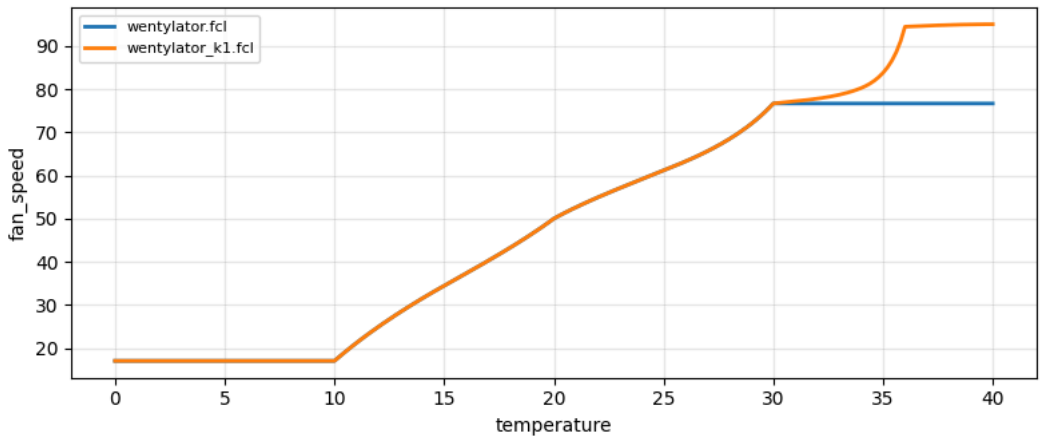


3. Testy sterownika

Zadanie 2C. Porównaj oryginał i Twoją wersję dla temperatur od 0 do 40 °C co 5 stopni. Wykres pokazuje prędkość dla wszystkich temperatur naraz.

```
TEMPERATURY = [0, 5, 10, 15, 20, 25, 30, 35, 40]
tabela(["wentylator.fcl", "wentylator_k1.fcl"], "temperature", TEMPERATURY)
krzywa(["wentylator.fcl", "wentylator_k1.fcl"], "temperature")
```

```
 temperature  wentylator  wentylator_k1
           0       17.03          17.03
           5       17.03          17.03
          10       17.03          17.03
          15       34.44          34.44
          20       50.00          50.00
          25       61.15          61.15
          30       76.67          76.67
          35       76.67          83.77
          40       76.67          95.03
```



Pytanie 3a. Co robi oryginalny sterownik powyżej 30 °C? Co zmieniło dodanie `very_hot`?

Pytanie 3b. Jaką prędkość daje sterownik przy 0 °C, a jaką przy 40 °C? Czy wentylator kiedykolwiek się wyłącza albo osiąga 100%? Dlaczego?

Odpowiedź do pytania 3a: Oryginalny sterownik powyżej 30 °C utrzymuje stałą wartość obrotów wentylatora (nieco mniej niż 80%). Dodanie very_hot sprawiło, że sterownik po przekroczeniu dalej zwiększa obroty, dążąc do maksimum.

Odpowiedź do pytania 3b: Przy temperaturze 0 °C sterownik narzuca ~17% prędkości obrotów, przy 40 °C sterownik (ten z zaimplementowanym very hot) narzuca ~95% prędkości obrotów. Wentylator nigdy nie osiąga 100%, ani nie wyłącza się, ponieważ zastosowana metoda wyostrzania to COG (środek ciężkości). Oblicza ona środek ciężkości całej figury powstałej z sumowania przyciętych zbiorów wyjściowych. Ponieważ zbiory te mają określone szerokości i nachodzą na siebie, ich wspólny środek ciężkości dla krańcowych

wartości wejściowych nigdy nie przesunie się na skrajne krawędzie przedziału (0% i 100%), lecz zawsze będzie ściągany w stronę środka przez pozostałą część obszaru funkcji przynależności.

---

4. Trzy konfiguracje

Zadanie 3A. Przetestuj sterownik z innymi ustawieniami. Każda konfiguracja to osobny plik FCL, w którym zmieniasz **jedną** rzecz:

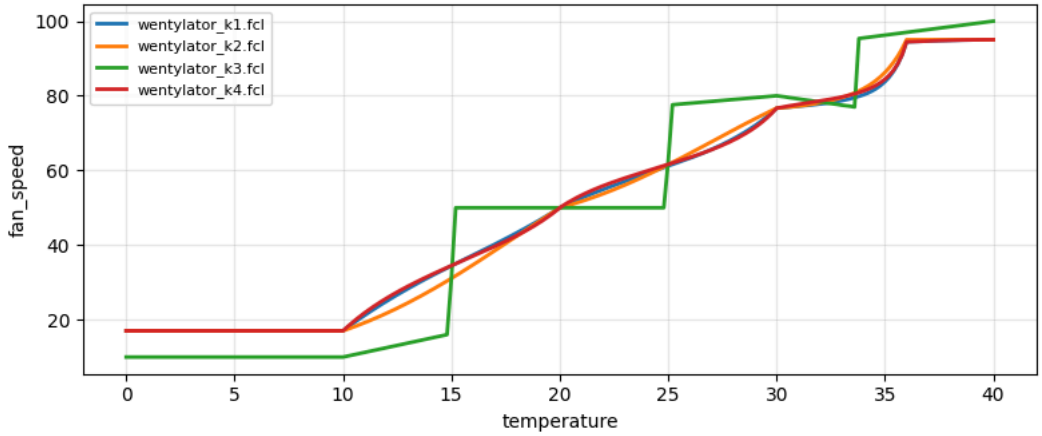
ustawienie w FCL	co określa	wariant do sprawdzenia
`ACT`	implikacja: jak siła reguły zmienia jej wniosek	`PROD` zamiast `MIN`
`METHOD`	wyostrzanie: jak z kształtu zrobić jedną liczbę	`MM` (środek maksimum) zamiast `COG`
`ACCU`	suma zbiorów na wyjściu: jak łączyć wnioski reguł	`BSUM` (suma ograniczona) zamiast `MAX`

Funkcja `wariant` tworzy kopię pliku ze zmienionym ustawieniem. Możesz też skopiować plik ręcznie i zmienić jedną linijkę.

```
# TODO: utwórz trzy konfiguracje (po jednej zmianie każda)
k2 = wariant("wentylator_k1.fcl", "wentylator_k2.fcl", ACT="PROD")
k3 = wariant("wentylator_k1.fcl", "wentylator_k3.fcl", METHOD="MM")
k4 = wariant("wentylator_k1.fcl", "wentylator_k4.fcl", ACCU="BSUM")

KONFIGURACJE = ["wentylator_k1.fcl", k2, k3, k4]   # TODO: dopisz k3 i k4
tabela(KONFIGURACJE, "temperature", TEMPERATURY + [28, 32, 34])
krzywa(KONFIGURACJE, "temperature")
```

```
Zapisano wentylator_k2.fcl: ACT = PROD
Zapisano wentylator_k3.fcl: METHOD = MM
Zapisano wentylator_k4.fcl: ACCU = BSUM
 temperature  wentylator_k1  wentylator_k2  wentylator_k3  wentylator_k4
           0          17.03          17.03          10.00          17.03
           5          17.03          17.03          10.00          17.03
          10          17.03          17.03          10.00          17.03
          15          34.44          31.08          30.93          34.50
          20          50.00          50.00          50.00          50.00
          25          61.15          61.54          62.23          61.60
          30          76.67          76.67          80.00          76.67
          35          83.77          86.25          96.25          84.43
          40          95.03          95.03         100.00          95.03
          28          68.45          70.79          79.00          68.08
          32          77.78          78.09          78.35          78.62
          34          80.24          81.90          95.50          81.26
```



Zadanie 3A (opis). Dla każdej konfiguracji opisz: co zmieniłeś, przy jakich temperaturach wynik się zmienił i o ile. Która zmiana ma największy wpływ?

- **wentylator_k2 (ACT=PROD):** Zmiana implikacji z obcinania (MIN) na skalowanie (PROD). Wynik zmienił się w punktach pośrednich przejściowych (np. przy 15 °C spadek z 34.44% do 31.08%, a przy 35 °C wzrost z 83.77% do 86.25%). Przebieg stał się bardziej zaokrąglony i łagodniejszy.
- **wentylator_k3 (METHOD=MM):** Zmiana metody wyostrzania na środek maksimum. Największa różnica ze wszystkich konfiguracji. Wykres stał się skokowy i schodkowy (np. stałe 10% do 10 °C, nagły skok do 50% przy 15 °C, osiąga pełne 100% przy 40 °C).
- **wentylator_k4 (ACCU=BSUM):** Zmiana sumowania zbiorów na sumę ograniczoną. Wyniki zmieniły się minimalnie, głównie przy wyższych temperaturach (np. przy 35 °C nieznaczny wzrost z 83.77% do 84.43%).

Największy wpływ: Zdecydowanie zmiana metody wyostrzania na MM (`wentylator_k3`), która całkowicie zlikwidowała płynność regulacji, tworząc skokowe progi działania.

---

5. Ocena sterownika

Zadanie 3B. Jak oceniasz działanie sterownika? Odnieś się do:

- czy prędkość rośnie z temperaturą płynnie, czy skokami,
- czy zakres prędkości (najmniejsza i największa) jest sensowny dla wentylatora,

- którą konfigurację wybrałbyś do prawdziwego urządzenia i dlaczego.

- Płynność: W podstawowych wersjach (k1, k2, k4) prędkość rośnie bardzo płynnie, bez gwałtownych skoków. W wersji k3 (wyostrzanie MM) zmiana zachodzi skokowo, schodkowo, co jest niekorzystne dla komfortu i trwałości silnika.
- Zakres: Zakres prędkości od ok. 17% do 95% jest bardzo sensowny. Minimalne obroty (17%) zapobiegają przegrzaniu silnika przy niskim napięciu, a maksymalne (95%) chronią przed przeciążeniem i nadmiernym hałasem.
- Wybór konfiguracji: Wybrałbym konfigurację wentylator_k2 (ACT=PROD). Skalowanie zamiast obcinania daje łagodniejsze przejścia obrotów przy zmianach temperatury, co przekłada się na cichszą i dłuższą pracę urządzenia.

---

6. Drugie wejście: wilgotność

Przy wysokiej wilgotności to samo ciepło jest bardziej uciążliwe, więc wentylator powinien kręcić się szybciej. Dodajemy do Twojego sterownika z części 2 drugie wejście `humidity` (wilgotność powietrza, 0–100%).

6.1 Rozszerzenie pliku

Nie musisz przepisywać całego pliku. Funkcja `rozszerz` bierze `wentylator_k1.fcl` i wkleja we właściwe miejsca trzy fragmenty, które piszesz w FCL:

1. nowe wejście w `VAR_INPUT` (gotowe),
2. blok `FUZZIFY humidity` z trzema stanami: `dry`, `normal`, `humid`,
3. co najmniej dwie nowe reguły, np. „ciepło **i** wilgotno ⟶ szybko", „gorąco **i** wilgotno ⟶ maksimum".

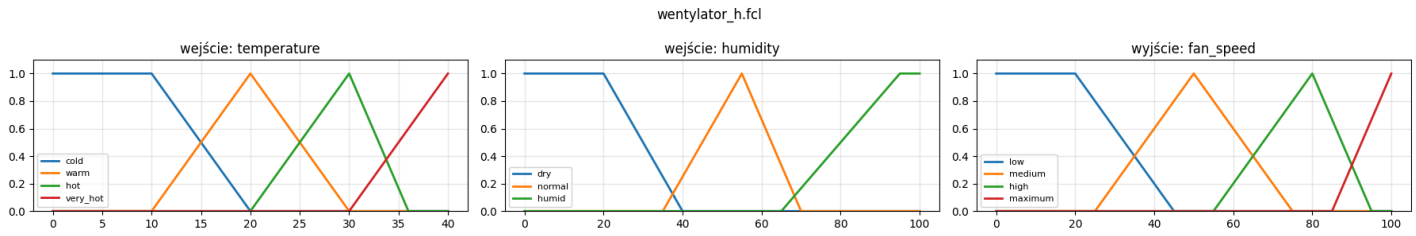
```
NOWE_WEJSCIE = "humidity : REAL;     (* wilgotność [%] *)"

# TODO: uzupełnij punkty (x, μ) – sąsiednie stany muszą się nakładać
NOWY_FUZZIFY = """
FUZZIFY humidity
    RANGE := (0 .. 100);
    TERM dry    := (0, 1) (20, 1) (40, 0);
    TERM normal := (35, 0) (55, 1) (70, 0);
    TERM humid  := (65, 0) (95, 1);
END_FUZZIFY
"""

# TODO: dopisz co najmniej dwie reguły z humidity (numeruj dalej: RULE 5, RULE 6, ...)
NOWE_REGULY = """
RULE 5 : IF temperature IS hot  AND humidity IS dry    THEN fan_speed IS medium;
RULE 6 : IF temperature IS hot  AND humidity IS normal THEN fan_speed IS high;
RULE 7 : IF temperature IS hot  AND humidity IS humid  THEN fan_speed IS maximum;
"""

rozszerz("wentylator_k1.fcl", "wentylator_h.fcl",
         wejscie=NOWE_WEJSCIE, fuzzify=NOWY_FUZZIFY, reguly=NOWE_REGULY)
rysuj_zbiory("wentylator_h.fcl")
```

```
Zapisano wentylator_h.fcl – plik wczytuje się poprawnie.
```



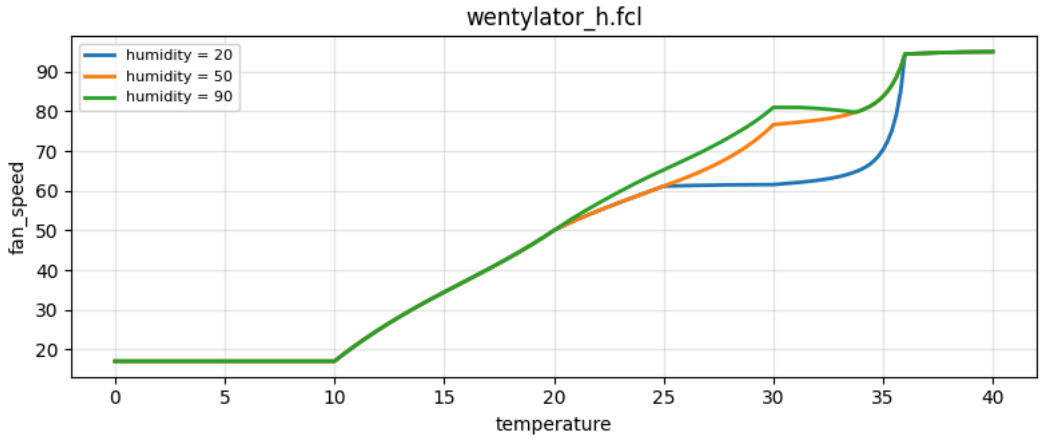
Testy. Ta sama temperatura przy suchym (20%) i wilgotnym (90%) powietrzu, a potem wilgotność od 0 do 100% przy stałych 25 °C.

```
print("Powietrze suche (humidity = 20):")
tabela("wentylator_h.fcl", "temperature", TEMPERATURY, humidity=20)
print("\nPowietrze wilgotne (humidity = 90):")
tabela("wentylator_h.fcl", "temperature", TEMPERATURY, humidity=90)
print("\nStała temperatura 25 °C:")
tabela("wentylator_h.fcl", "humidity", [0, 20, 40, 60, 75, 90, 100], temperature=25)
krzywe("wentylator_h.fcl", "temperature", "humidity", [20, 50, 90])
```

```
Powietrze suche (humidity = 20):
 temperature  wentylator_h
           0         17.03
           5         17.03
          10         17.03
          15         34.44
          20         50.00
          25         61.15
          30         61.54
          35         70.33
          40         95.03

Powietrze wilgotne (humidity = 90):
 temperature  wentylator_h
           0         17.03
           5         17.03
          10         17.03
          15         34.44
          20         50.00
          25         65.26
          30         81.01
          35         83.77
          40         95.03

Stała temperatura 25 °C:
    humidity  wentylator_h
           0         61.15
          20         61.15
          40         61.15
          60         61.15
          75         63.88
          90         65.26
         100         65.26
```



Pytanie 6a. O ile wilgotne powietrze zwiększa prędkość przy 25 °C? Czy to dużo, czy mało?

Pytanie 6b. Czy wilgotność cokolwiek zmienia przy 10 °C? Dlaczego?

Pytanie 6c. Reguła 2 („warm → medium") nic nie wie o wilgotności. Co robi, gdy powietrze jest wilgotne? Jak to wpływa na wynik z pytania 6a?

- **Odpowiedź 6a:** Przy 25 °C wilgotne powietrze (90%) zwiększa prędkość z 61.15% do 65.26% (czyli zaledwie o ok. 4 punkty procentowe). To zdecydowanie mało, zmiana jest niemal nieodczuwalna.
- **Odpowiedź 6b:** Przy 10 °C wilgotność nic nie zmienia (prędkość zostaje na poziomie 17.03%). Dzieje się tak, ponieważ przy 10 °C aktywna jest tylko reguła 1 dla zimnego powietrza (`cold`), która nie uwzględnia poziomu wilgotności.
- **Odpowiedź 6c:** Gdy powietrze jest wilgotne, reguła 2 nadal aktywuje się w pełni (dla 25 °C częściowo) i dąży do ustawienia prędkości `medium`. Ponieważ wciąż ma duży wpływ na wynik końcowy, "ściąga" ona wyjściowy środek ciężkości w dół i blokuje mocniejsze działanie nowych reguł dla wilgotnego powietrza, przez co wzrost w 6a jest tak mały.

6.2 Poprawka: stare reguły tylko dla niewilgotnego powietrza

Zmień reguły 2 i 3 tak, żeby działały tylko, gdy powietrze **nie** jest wilgotne. FCL ma do tego słowo `NOT`:

`RULE 2 : IF temperature IS warm AND humidity IS NOT humid THEN fan_speed IS medium;`

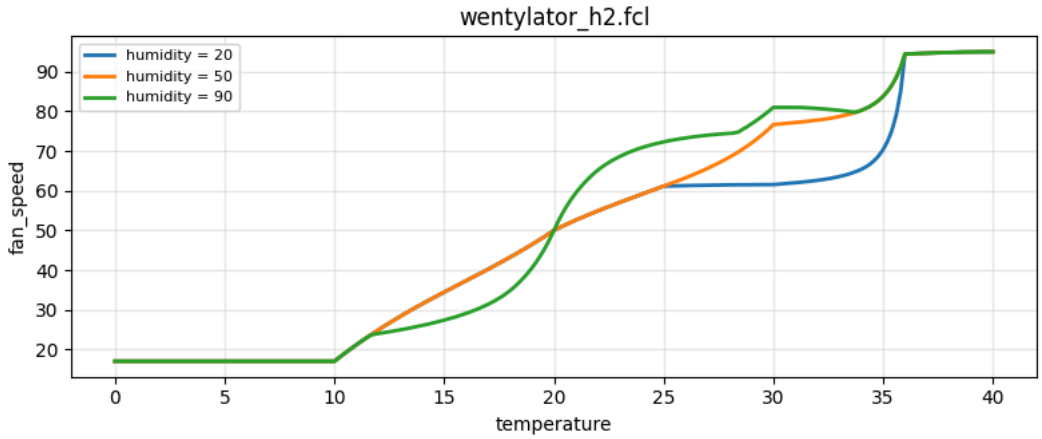
Komórka poniżej robi kopię pliku i zamienia w niej tekst reguł.

```
tekst = open("wentylator_h.fcl", encoding="utf-8").read()
# TODO: zamień reguły 2 i 3 (tekst przed zamianą musi być DOKŁADNIE taki jak w Twoim pliku)
tekst = tekst.replace("RULE 2 : IF temperature IS warm THEN fan_speed IS medium;",
                      "RULE 2 : IF temperature IS warm AND humidity IS NOT humid THEN fan_speed IS medium;")
tekst = tekst.replace("RULE 3 : IF temperature IS hot AND humidity IS NOT humid  THEN fan_speed IS high;",
                      "RULE 3 : IF temperature IS hot  THEN fan_speed IS high;")
open("wentylator_h2.fcl", "w", encoding="utf-8").write(tekst)

print("Stała temperatura 25 °C:")
```

```
tabela(["wentylator_h.fcl", "wentylator_h2.fcl"], "humidity", [0, 20, 40, 60, 75, 90, 100], temperature=25)
krzywe("wentylator_h2.fcl", "temperature", "humidity", [20, 50, 90])
```

```
Stała temperatura 25 °C:
    humidity  wentylator_h  wentylator_h2
           0         61.15          61.15
          20         61.15          61.15
          40         61.15          61.15
          60         61.15          61.15
          75         63.88          63.88
          90         65.26          72.31
         100         65.26          80.27
```



Pytanie 6d. Jak zmienił się wynik przy 25 °C i wilgotności 90% po poprawce? Dlaczego?

Pytanie 6e. Pełna baza reguł dla 4 stanów temperatury i 3 stanów wilgotności ma 4 × 3 = 12 reguł. Ile reguł ma Twój sterownik? Czy przy dwóch wejściach logika rozmyta daje coś, czego nie dałaby zwykła funkcja temperatury?

- **Odpowiedź 6d:** Wynik przy 25 °C i wilgotności 90% wzrósł po poprawce z 65.26% do 72.31%. Wynika to z faktu, że reguła dążąca do poziomu 'medium' została wyłączona (dzięki warunkowi 'NOT humid'), dając pełne pole działania nowym regułom zwiększającym obroty przy dużej wilgotności.
- **Odpowiedź 6e:** Nasz sterownik ma aktualnie 7 reguł. Przy dwóch wejściach logika rozmyta pozwala na intuicyjne i elastyczne modelowanie zachowań nieliniowych (np. zwiększona wrażliwość na temperaturę przy wysokiej wilgotności), co w przypadku zwykłej funkcji matematycznej wymagałoby tworzenia skomplikowanych formuł analitycznych.

Do oddania

1. Pliki FCL: `wentylator_k1.fcl` (z `very_hot`) oraz co najmniej dwa warianty (`wentylator_k2.fcl`, `wentylator_k3.fcl`, ...). W Colab: kliknij plik w panelu po lewej → „Pobierz”.
2. Pliki z wilgotnością: `wentylator_h.fcl` i `wentylator_h2.fcl`.
3. Ten notebook z wynikami i odpowiedziami zapisany jako PDF: Plik → Drukuj → Zapisz jako PDF.